AP CSA Math Class: Methods, Casts & Ranges

abs, pow, sqrt, random - return types, casts, ranges and the traps

`Math` is a utility class in `java.lang` (so no `import`). Every member is `static` - you call it on the class, never on an object, and `new Math()` does not compile. This page is the full reference behind section 8 of the method notes.

1. The Four Methods on the Quick Reference

Method signature	Returns	Example	Result
`static int abs(int x)`	`int`	`Math.abs(-7)`	`7`
`static double abs(double x)`	`double`	`Math.abs(-2.5)`	`2.5`
`static double pow(double base, double exponent)`	`double`	`Math.pow(2, 3)`	`8.0`
`static double random()`	`double` in `[0.0, 1.0)`	`Math.random()`	e.g. `0.7314`
`static double sqrt(double x)`	`double`	`Math.sqrt(16)`	`4.0`

Only `abs` can return an `int`. `pow`, `random`, and `sqrt` always return `double`, which is why they need a cast before they can be stored in an `int`.

```
int a = Math.abs(-7);          // 7        int overload, no cast needed
int b = (int) Math.sqrt(16);   // 4        cast required: double -> int
int c = (int) Math.pow(2, 8);  // 256      cast required
int d = Math.abs(-7.5);        // CE: double result assigned to an int
```

2. abs: Two Overloads, One Edge Case

The compiler picks `abs(int)` or `abs(double)` from the argument's **static type**:

Argument	Chosen overload	Returns	Note
`int` or `char`	`abs(int)`	`int`	`Math.abs('A' - 'C')` is `2`
`double` or `float`	`abs(double)`	`double`	`Math.abs(-3.0)` is `3.0`
`Math.abs(-0.0)`	`abs(double)`	`0.0`	Negative zero is normalised

(extra) The overflow edge case, for the curious: `Math.abs(Integer.MIN_VALUE)` is **negative**. The true answer `2147483648` does not fit in an `int`, so it wraps back to `-2147483648`. You will not be asked to fix this, but it explains why "abs is never negative" is technically false.

```
int diff = Math.abs(a - b);        // distance between two ints
double gap = Math.abs(x - y);      // distance between two doubles
boolean close = Math.abs(x - y) < 0.000001;   // the safe way to compare doubles
```

3. pow: Always a `double`

- Both arguments are `double`; `int` arguments are promoted automatically.
- The result is `double` **even when it comes out whole**: `Math.pow(3, 2)` is `9.0`.
- Negative exponents are legal: `Math.pow(2, -1)` is `0.5`.
- Anything to the zero power is `1.0`, including `Math.pow(0, 0)`.
- Huge results become `Infinity` rather than throwing an exception.

```
double area = Math.pow(side, 2);          // side squared
int square = (int) Math.pow(n, 2);        // cast when an int is required
double growth = 1000 * Math.pow(1.05, 10); // compounding
```

Integer division happens before the call. Arguments are evaluated first, so:

```
Math.pow(5 / 2, 2)     // 4.0    because 5 / 2 is 2
Math.pow(5 / 2.0, 2)   // 6.25   because 5 / 2.0 is 2.5
```

4. `sqrt`: `double`, and `NaN` for Negatives

Call	Result	Why
`Math.sqrt(16)`	`4.0`	Perfect squares still come back as `double`
`Math.sqrt(2)`	`1.4142135623730951`	Irrational, so never compare with `==`
`Math.sqrt(0)`	`0.0`	Fine
`Math.sqrt(-4)`	`NaN`	"Not a number"; every comparison with `NaN` is `false`

```
double d = Math.sqrt(Math.pow(dx, 2) + Math.pow(dy, 2));  // distance formula
double side = Math.sqrt(area);                            // inverse of squaring
if (area >= 0) { ... }                 // guard: sqrt(<0) is NaN
```

`Math.sqrt(9) == 3` is `true` (the `int` is promoted to `3.0`), but `Math.sqrt(2) * Math.sqrt(2) == 2` is `false`. Compare doubles with a tolerance instead.

5. `random`: The Range Formulas

`Math.random()` returns a `double` in `[0.0, 1.0)`: it can return `0.0`, it can never return `1.0`. Every random integer you need is built by scaling, casting, then shifting.

Goal	Formula	Possible values
Die roll	`(int) (Math.random() * 6) + 1`	1-6
Array index	`(int) (Math.random() * arr.length)`	0 to length-1
Coin flip (0 or 1)	`(int) (Math.random() * 2)`	0, 1
Any int `min`..`max`	`(int) (Math.random() * (max - min + 1)) + min`	inclusive both ends
Double `0.0`..`1.0`	`Math.random()`	no cast, no arithmetic

```
int die   = (int) (Math.random() * 6) + 1;      // 1..6, not 0..5
int idx   = (int) (Math.random() * names.length);
int grade = (int) (Math.random() * 41) + 60;    // 60..100
double p  = Math.random();                      // [0.0, 1.0)
```

Three ways this goes wrong

```
int bad1 = (int) Math.random() * 10;      // ALWAYS 0: the cast hits the fraction first
int bad2 = (int) (Math.random() * 10) + 1; // 1..10, off by one if you wanted 0..9
int bad3 = (int) Math.random() * 6 + 1;   // still 0 * 6 + 1 = 1, every single time
```

The cast must wrap the **entire product**, and the `+ 1` goes **outside** the cast when you want to shift the range up.

6. Rounding: Four Answers for the Same Number

Value	`(int) x` (truncate)	`Math.round(x)`	`Math.floor(x)` *(extra)*	`Math.ceil(x)` *(extra)*
`2.5`	`2`	`3`	`2.0`	`3.0`
`2.4`	`2`	`2`	`2.0`	`3.0`
`-2.5`	`-2`	`-2`	`-3.0`	`-2.0`
`-2.4`	`-2`	`-2`	`-3.0`	`-2.0`

- `(int)` **truncates toward zero** - it is not rounding.

- `Math.round(double)` returns a **`long`**; assigning it to an `int` needs `(int) Math.round(x)`. `Math.round(float)` returns `int` instead.
- `Math.round(-2.5)` is `-2`, not `-3`: it rounds half **up** (toward positive infinity), which surprises people on negative numbers.
- `floor` / `ceil` return `double`, so they usually need a cast as well.

```
int rounded = (int) Math.round(87.6);     // 88
int truncated = (int) 87.6;               // 87
int average = (int) Math.round(sum / (double) count);   // the usual "round the average"
```

7. What the Exam Actually Asks You to Do With It

Task	Expression
Distance between two points	`Math.sqrt(Math.pow(x2 - x1, 2) + Math.pow(y2 - y1, 2))`
Difference / deviation	`Math.abs(actual - expected)`
Area or volume of a square/cube	`Math.pow(side, 2)` / `Math.pow(side, 3)`
Monthly compounding	`principal * Math.pow(1 + rate / 12, months)`
Random element of an array	`arr[(int) (Math.random() * arr.length)]`
Round to nearest int	`(int) Math.round(value)`
Circle area	`Math.PI * Math.pow(r, 2)` *(extra: `Math.PI` is a field, no parentheses)*

8. Traps

Written	What happens	Correct form
`int n = Math.pow(2, 3);`	`CE: incompatible types` (double into int)	`(int) Math.pow(2, 3)`
`(int) Math.random() * 10`	Always `0`	`(int) (Math.random() * 10)`
`int x = Math.sqrt(9);`	`CE: incompatible types`	`(int) Math.sqrt(9)`
`new Math()`	`CE: Math() has private access`	Call the static methods on the class
`Math.PI()`	`CE: cannot find symbol`	`Math.PI` - a field, not a method
`Math.abs(-7.5)` into an `int`	`CE: incompatible types`	`(int) Math.abs(-7.5)`
`Math.pow(5 / 2, 2)` expecting `6.25`	`4.0`	`Math.pow(5 / 2.0, 2)`
`Math.sqrt(2) * Math.sqrt(2) == 2`	`false`	Compare with a tolerance
`Math.round(2.5)` into an `int`	`CE` (`long` result)	`(int) Math.round(2.5)`
`myMath.pow(2, 3)` on an object	`CE`	`Math.pow(2, 3)`
`Math.random()` used bare as an `int`	`CE: incompatible types`	Cast the scaled product

9. Before You Submit

- [ ] Every `Math.pow` / `sqrt` / `random` result that lands in an `int` is cast.
- [ ] The cast wraps the whole product, not just `Math.random()`.
- [ ] The `+` / `-` shift sits **outside** the cast when the range does not start at 0.
- [ ] Range check: die is 1-6 (not 0-5), index is 0 to length-1 (never `length`).
- [ ] Division inside `pow` / `sqrt` was checked for accidental integer division.
- [ ] `abs` was used for distance, not `a - b` (which can be negative).
- [ ] Rounding used `Math.round` where "nearest" was asked for, `(int)` only where truncation was intended.
- [ ] Nothing computes `Math.sqrt` of a value that could be negative without a guard.